

OpenLoopEvolve: 面向长时程复杂任务的循环策略可验证自进化框架

Siqi Wang*, Xinlin Li*, Zhenglin Li, and Li Li

摘要——长时程复杂任务要求智能体在持续变化的环境中反复观测状态、制定计划、调用工具、验证结果并从失败中恢复。然而，此类控制经验往往局限于单一上下文或固定提示，难以在历史轨迹中积累和复用。本文提出 OpenLoopEvolve (OLE)，一个以循环策略为核心的自进化框架。OLE 将智能体的观测、规划、记忆、动作、验证、恢复、停止和预算控制表示为具有版本和谱系的可移植策略资产，并提供可根据实际需求选择的在线和离线进化模式：在线模式基于持续运行的反馈触发候选生成，而离线模式则从归档轨迹和失败证据中搜索候选策略。两种模式共享同一进化机制，包括大语言模型的自主提案、冠军-挑战者配对评估以及稳健发布。在线发布的策略在后续任务边界激活，利用后续反馈进行监控，并在满足退化条件时回滚至其父版本。在模拟业务基准 YC-Bench 上，与固定的初始循环策略相比，两种模式均提升了总体任务性能、任务成功率和风险度量。结果表明，将循环策略视为可治理资产能够支持控制经验的积累、比较、发布和复用，并提升智能体在长时程复杂任务上的性能。

索引词——大语言模型智能体，长时程任务，循环策略，自进化

1. 引言

大语言模型智能体正从生成一次性答案的工具，演变为能够在外部环境中持续行动的任务执行者。在长时程复杂任务的一系列决策中，智能体必须获取观测、修订计划、调用工具、验证结果，并在局部失败后决定是重试、恢复还是停止。推理与行动的交替使智能体能够根据环境反馈调整行为 [1]，而交互式评估已开始从短流程扩展到跨页面操作和有状态业务 workflow [2] – [5]，以及 YC-Bench 所研究的模拟年度企业管理 [6]。在这些任务中，过早停止可能留下未经验证的工件，而反复规划与

无效调用会消耗后续所需的资源；此外，一个动作引起的状态变化可能使原计划中的后续步骤失效。因此，观察粒度、恢复方法和预算分配形成相互依赖的控制关系，最终结果取决于整个执行循环，而非仅取决于单步生成质量。现有研究利用外部记忆、语言反思和经验提取，在后续决策中重用过去的事件或失败 [7] – [9]；可执行技能库进一步支持程序性行为的检索与组合 [10]。与此同时，声明式流水线、基于代码的工作流和模块化智能体将模型外部的结构引入自动化优化 [11] – [14]，轨迹反馈也被用于持续修正多步推理过程 [15]。这些方向分别改进了经验内容和执行结构，但尚未将观察、规划、记忆使用、验证、恢复、停止和预算分配之间的整体控制关系表示为一个统一对象。记忆可以提供过去的内容，技能可以复现局部操作，但二者本质上都没有规定何时调用经验、如何检查执行结果或何时终止。尽管历史轨迹可以被存储或检索，但其成功与失败仍难以转化为对整个循环的可比较、可迁移的修改。最近兴起的循环工程视角将触发器、目标、验证、停止规则和记忆组织视为模型外部循环的设计元素 [16]，为上述问题提供了新的分析层次。一个能够跨运行累积的循环对象还需要稳定的控制语义、宿主接口和版本关系，以便相同的修改在不同运行中具有可比较的含义。循环优化还必须将候选生成与正式发布分离：在没有候选验证的情况下立即重写循环，可能会给后续任务引入偶然噪声或尾部退化，而始终保留固定循环则使系统无法从反复出现的失败中学习。关于证据门控的研究强调了生成声明与验证状态之间的区别 [17]；冠军 – 挑战者机制也在预测模型管理和在线自动机器学习中通过候选比较来约束版本替换 [18], [19]。然而，这些机制尚未直接应用于完整的智能体循环，也没有为离线历史轨迹和在线运行反馈提供共享的策略资产和发布语义。在此背景下，本文聚焦于两个相关问题：第一，如何将完整的执行循环分离出来

* 王思琪与李新林对本文贡献等同，并列为第一作者

作者信息。王思齐和李力任职于清华大学自动化系（电子邮箱：wang-sq24@mails.tsinghua.edu.cn；lili@mail.tsinghua.edu.cn）。李欣林任职于清华大学工业工程系（电子邮箱：lix122@mails.tsinghua.edu.cn）。李正林任职于清华大学深圳国际研究生院（电子邮箱：li-zl23@tsinghua.org.cn）。通讯作者：李力。

OpenLoopEvolve Framework

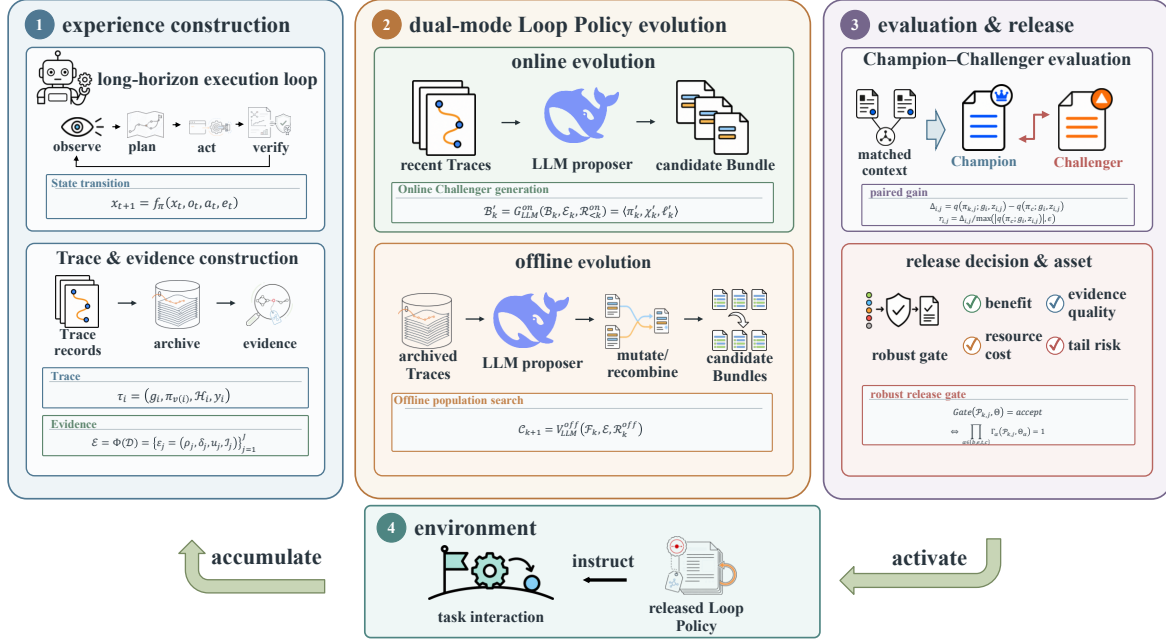


图 1. OpenLoopEvolve 总体框架。已发布的循环策略在后续任务边界被激活，进入宿主环境，并在长时程任务中控制交互，产生可归因的迹。这些迹在共享证据契约下转换为演化证据，进入在线或离线循环策略演化。候选策略在与冠军策略相同的条件下进行评估，通过稳健门控后，作为带版本的策略资产发布，并在任务边界激活。

来自宿主实现和单一上下文，以形成可版本化、可移植、可比较和可重用的外部策略资产；其次，如何利用历史迹和当前反馈自主改进该资产，同时仅允许通过候选验证、已发布并已在任务边界激活的策略影响未来运行。在线更新需要避免干扰当前正在执行的任务状态，而离线搜索需要从多个异构迹中形成可比较的候选评估。因此，两种模式在证据可用时间上不同，但需要一致的资产和发布语义。第一个问题决定了作为循环演化单元的对象，第二个问题决定了证据如何驱动策略变化以及这些变化何时可以发布和激活。本文的目标不是允许智能体在单次运行中任意重写自身，而是将策略搜索置于执行路径之外，使候选策略、评估证据和正式版本保持清晰分离。

读取、比较并独立于原始迹（Trace）进行复用。一次运行的迹随后被提炼为可追溯的证据，指向策略修改，之后由大语言模型（Large Language Model）自主提出候选方案。OLE 根据证据来源提供在线和离线两种演化模式。在线模式基于近期反馈生成单个挑战者（Challenger）；经过配对评估和稳健门控后，挑战者作为后继资产发布，在后续任务边界激活，并接受退化监控与回滚。离线模式从归档迹中构建候选种群，通过批量评估、精英保留以及自主变异和重组进行多代搜索。两种模式可根据运行连续性、预算和候选验证时机分别采用，同时共享“证据—候选—候选验证—发布—激活”的边界：候选生成属于搜索过程，只有满足候选验证要求的策略才能发布，且只有已在任务边界激活的已发布策略才能进入执行路径。这形成了从历史交互到后续策略的闭环，如图 1 所示，使控制经验能够持续演化，同时防止临时候选直接改变执行路径。本文的主要贡献总结如下：

为解决这些问题，本文提出 OpenLoopEvolve（OLE），一个用于长时程复杂任务中循环策略的可验证自演化框架。OLE 将观察、规划、记忆、动作、验证、恢复、停止和预算控制组织为外部化的循环策略，并将策略、其适用条件和版本来源封装在策略谱系中的 Bundle 内。循环策略通过通用控制语义约束运行，而不绑定到宿主的特定模型和工具实现，使循环经验能够

- 1) 本文提出循环策略与束协议（Loop Policy and Bundle Protocol），将完整的循环控制表示为具有显式接口、版本和谱系的外部策略资产，

使历史控制经验能够在多次运行间被保存、比较、交换和复用。

- 2) 本文提出基于迹证据的在线与离线自演化方法。大语言模型自主提出候选方案，而配对评估、稳健发布、任务边界激活和回滚共同约束其进入执行路径。据我们所知，本工作首次将冠军—挑战者（Champion – Challenger）发布机制应用于外部化循环策略的离线和在线自演化。
- 3) 本文在 YC-Bench 上评估 OLE。使用固定模型、基准配置和官方种子获得的结果表明，在线和离线模式在长期回报、任务成功率、年度存活率和风险度量方面均优于固定的初始循环策略。

本文的其余部分组织如下。第二节回顾了长程交互智能体、记忆与轨迹经验的复用、智能体工作流优化以及自我进化的相关研究。第三节定义了长程复杂任务、循环交互过程、迹、进化证据、循环策略和束。第四节介绍了 OpenLoopEvolve 的共享进化框架及其在线和离线实现。第五节给出了实验结果与分析。第六节对全文进行了总结。本文的代码可在 <https://github.com/yoyoshikc/OpenLoopEvolve> 找到。

II. 相关工作

A. 长程交互智能体与循环执行

与环境的多轮交互是大语言模型智能体在复杂任务中的一项基本能力。与一次性答案生成不同，智能体必须接收环境状态、选择动作，并利用新的反馈在一系列决策中推进其目标；因此，任务性能取决于跨步骤的一致推理和执行。现有的综合基准已经在多样化的交互环境中检验了这些能力，并进一步记录了智能体在任务完成前的中间进展 [2], [3]。关于这一过程的一条研究路线构建了一个执行循环，其中推理和动作交替进行。Yao 等人 [1] 将语言推理、外部动作和环境观察组织在同一条轨迹中，使模型能够根据执行结果调整后续判断。另一条研究路线将此类循环嵌入涉及真实状态变化的工作流中：Zhou 等人 [4] 在网页环境中评估跨页面操作和功能正确性，而 Yao 等人 [5] 进一步评估了涉及工具、用户和领域状态的交互中的任务完成情况。相应地，长程基准也开始强调控制决策的累积后果：He 等人 [6] 通过持续一年的模拟业务运营来测试长期规划和一致执行，而不是仅评估单轮响应。这些研究建立了长程任务的闭环交互范式，并展示了规划、工具使用和状态反馈在任务完成中的作用。它们的主要

然而，目标仍是在给定的执行结构内改进或评估智能体行为；观察如何组织、规划或恢复何时发生、结果如何验证以及执行何时停止，通常由特定方法或宿主框架决定。这为研究如何将这些控制规则与单次运行的实现细节分离，并将其视为统一对象进行比较和优化留下了空间。使循环控制规则能够跨运行持续改进，还需要解决如何积累历史交互并将其重新引入后续决策的问题。

B. 记忆、反思与轨迹经验复用

记忆与反思为智能体跨轮次积累经验提供了主要手段。其共同前提是在外部介质中保存超出有限上下文窗口的历史交互，并在后续决策中检索、总结或重组这些交互，从而维持长期行为的一致性。Park 等人 [7] 将经验记录与反思和规划相连接，使智能体能够从过去事件中提炼更高层次的洞见，并利用这些洞见指导后续行为；这表明历史轨迹不仅能作为原始对话记录，还能为持续决策做出贡献。现有方法在保存和利用经验的方式上存在差异。Shinn 等人 [8] 将任务反馈转化为语言反思并存储在情景记忆中，使后续试验能够借鉴失败经验而无需更新模型参数；Zhao 等人 [9] 从一组训练任务中提取自然语言经验，并在推理时联合检索抽象洞见和相似的成功轨迹。与文本经验不同，Wang 等人 [10] 将从环境反馈中生成的成功程序存储在可检索的技能库中，使程序化行为能够在新的《我的世界》世界中被组合和重用。这些方法分别表明，历史轨迹可以被提炼为反思性文本、经验知识或可执行技能。这些机制显著扩展了智能体重用过往经验的能力，但其更新目标主要是记忆内容、语言经验或任务技能，通常不直接表示观察、规划、验证、恢复、停止和预算分配之间的整体控制关系。本文并不试图增加另一种形式的记忆内容；相反，本文从支持策略比较的迹中提炼证据，并利用这些证据修改完整循环的控制方式。这一区别进一步引出了智能体工作流及其自演化的问题。

C. 智能体工作流优化与自演化

随着智能体系统从单一提示扩展为包含模型调用、工具和控制流的复合程序，研究对象已逐渐从提示优化转向工作流与智能体结构的优化。Khattab 等人 [11] 将语言模型流水线表示为可组合的声明式模块，并编译其

针对指定度量指标对参数进行评估；Hu 等人 [12] 则将智能体系统表示为代码，使元智能体能够生成并搜索新的智能体程序，其代码空间原则上可涵盖提示、工具使用模式及 workflow 组合。这些研究表明，模型外部的执行结构本身可以成为自动优化的对象。就优化所依据的证据而言，现有工作可归纳为两条路径：离线结构搜索与由运行时反馈驱动的持续改进。离线搜索利用现有任务、执行评估及累积的搜索经验来生成候选结构：Zhang 等人 [13] 将代码表示的智能体 workflow 建模为搜索空间，并利用执行反馈和树状经验迭代修改 workflow；Shang 等人 [14] 将规划、推理、工具使用和记忆抽象为具有统一接口的模块，并通过进化和重组搜索智能体设计。相比之下，轨迹驱动的自进化研究更强调利用新生成的交互反馈来调整后续求解过程；Guo 等人 [15] 从多步推理轨迹中提取反馈，并迭代优化推理过程。前者侧重于在受控评估环境中搜索结构，后者侧重于让运行时经验持续影响后续行为；但其重点仍是推理过程的迭代优化，尚未涉及部署中的策略发布治理。这两条路径共同推动智能体从固定设计走向经验驱动的持续优化。近期出现的循环工程进一步将注意力集中在模型外部的循环结构上。Macedo [16] 将触发条件、目标、验证、停止规则和记忆组织为可复用的循环规范，而 Huang 等人 [17] 强调只有通过验证门控的证据才能驱动智能体生命周期中的状态转换。这些研究将 workflow 优化扩展到完整执行循环的系统化设计，并开始处理其中的验证和状态治理。在相关工作方面，冠军-挑战者范式已被用于生产预测模型的持续比较和替换 [18]，并进一步发展成为一种超参数选择算法，同时维护一个冠军模型和多个在线挑战者 [19]。然而，现有方法主要比较预测模型或学习配置。循环工程本身尚未建立统一的对象边界和接口标准；大多数现有优化结果仍是特定程序、任务结构或框架内部配置，而离线搜索和在线运行时反馈仍缺乏共享的策略资产和发布语义。为弥补这一差距，本文将循环控制逻辑表示为版本化且可移植的循环策略，使用统一协议兼容离线和在线候选方案，并通过冠军-挑战者比较、证据约束发布和回滚来治理其演化。据我们所知，本文首次将冠军-挑战者发布机制应用于智能体的离线和在线自演化。

外部化循环策略，使长程任务中的循环控制经验能够作为独立资产被验证、发布和复用。

三、问题定义

一、长程任务与循环控制问题

在交互式环境中持续执行多轮任务是智能体推理、决策和执行能力的重要考验，已成为通用智能体研究与评估的主要焦点 [2]、[3]。此类任务要求智能体在连续决策轮次中与外部环境进行闭环交互，以达成给定目标：智能体根据当前观测选择动作，动作改变环境状态并产生新的反馈，后续决策因此依赖于先前的交互。现有研究主要从两个层面刻画这些任务的长程特性：求解机制和评估协议。在求解机制层面，Yao 等人 [1] 将任务执行组织为闭环交互，使智能体能够根据反馈更新计划并处理异常。在评估协议层面，WebArena 和 Tau-bench [4]、[5] 将任务构建为具有多步依赖关系的真实交互式 workflow，并根据功能正确性或执行后的环境状态判定任务完成情况。前者反映了执行过程中的闭环决策和状态依赖，后者则确保任务结果能够通过外部环境进行验证。遵循闭环交互的视角，我们将长程复杂任务的求解视为一个执行循环，智能体在该循环中反复经历推理、动作和环境观测，并重点关注该循环中控制观测、规划、动作、验证、恢复和停止的机制。我们将长程复杂任务定义为：其求解过程无法可靠地简化为单次模型生成而不与环境交互，而是要求智能体在有限资源约束下执行多个具有状态依赖性的连续决策步骤，根据工具返回和环境反馈调整后续行为，直至产生可验证的任务结果。形式上，我们使用以下可执行契约来描述长程复杂任务：

$$g = \langle d, \mathcal{U}_g, \mathcal{M}_g, \mathcal{S}_g, \mathcal{V}_g, \beta_g \rangle, \quad g \in \mathcal{G}. \quad (1)$$

此处， $\mathcal{G}$ 表示长时程复杂任务的空间。 d 描述任务目标、预期工件及初始条件，为循环内的规划与重规划提供锚点。

$\mathcal{U}_g$ 指定任务允许的执行器集合，从而限制循环可用的工具调用与环境动作。 $\mathcal{M}_g$ 提供评估度量，用以衡量执行进度、结果质量与资源成本。 $\mathcal{S}_g$ 限制可支持判断的证据来源，如环境观察、运行日志与任务工件，防止仅凭模型自身陈述即认定任务完成。 $\mathcal{V}_g$

规定验证的时机与方式，并将其结果反馈至重规划、恢复与停止环节。 β_g 是资源预算向量，包含交互轮数、工具调用次数、时间与令牌数等的上界，从而约束重试与探索。这六个字段分别界定循环的目标、能力边界、度量准则、判断依据、验证机制与资源边界。基于此形式化定义，我们进一步刻画智能体在单次运行中的循环交互过程，以显式揭示观察、内部状态、动作与环境反馈之间的动态关系。在给出其结构化组件之前，我们使用 Π 抽象表示循环策略空间，并令 $\pi \in \Pi$ 表示约束当前执行循环的循环策略；其八组件结构定义于第 III-C 节。循环控制过程可抽象为

$$x_{t+1} = f_\pi(x_t, o_t, a_t, e_t), \quad a_{t+1} \sim h_\pi(x_{t+1}), \quad (2)$$

其中 t 表示当前交互步骤； $x_t \in \mathcal{X}$ 表示步骤 t ； $o_t \in \mathcal{O}$ 处的内部控制状态，表示智能体在步骤 t ； $a_t \in \mathcal{A}$ 从工具或环境接收到的观测，表示在步骤 t ； $\mathcal{E}_{\text{env}}$ 执行的动作，表示环境事件空间，且 $e_t \in \mathcal{E}_{\text{env}}$ 表示由动作 a_t ； π 引起的环境事件，表示约束循环行为的循环策略； f_π 表示更新函数，在循环策略 π 下，从 x_t, o_t, a_t 计算后继状态，且 e_t ； x_{t+1} 表示由 f_π ； h_π 产生的下一控制状态，表示从 x_{t+1} 选择动作的决策函数；且 a_{t+1} 表示由 h_π 产生的下一动作。动作可以是工具调用、信息获取、工件修改、结果验证或停止决策，其可用范围由任务契约中的 $\mathcal{U}_g$ 和宿主能力共同约束。状态 x_t 是运行循环所需控制信息的抽象表示，不必是完全可观测的马尔可夫状态。包括观测、规划、记忆使用、反思、风险处理、评估和预算分配在内的因素，通过 f_π 和 h_π 影响后续状态更新和动作选择。在刻画了循环交互过程之后，我们还必须明确评估循环策略的依据及其优化目标。令 $q(\pi; g, z) \in \mathbb{R}$ 表示循环策略 π 在任务 g 上、受控运行条件 $z \in \mathcal{Z}$ 下取得的价值得分，其中 $\mathcal{Z}$ 表示运行条件空间， q 由任务契约中的评估度量 $\mathcal{M}_g$ 和验证协议 $\mathcal{V}_g$ 决定。在这些约定下，循环策略优化寻求在相同任务和受控运行条件下，获得更高价值得分且满足验证要求和资源边界的循环策略。该目标可形式化为

$$\begin{aligned} \pi^* \in \arg \max_{\pi \in \Pi} \quad & q(\pi; g, z) \\ \text{s.t.} \quad & \mathbf{c}_{\mathcal{V}_g}(\pi; g, z) \preceq \kappa_g, \\ & \mathbf{b}(\pi; g, z) \preceq \beta_g, \end{aligned} \quad (3)$$

其中 Π 是第 III-C 节正式定义的循环策略空间， π^* 表示对此的最优循环策略

约束优化问题中， $\mathbf{c}_{\mathcal{V}_g}(\pi; g, z)$ 表示由验证协议确定的验证风险向量， $\mathcal{V}_g, \kappa_g$ 表示相应的可接受上界， $\mathbf{b}(\pi; g, z)$ 表示资源消耗向量， $\preceq$ 表示逐元素向量比较。该优化问题共同刻画了结果质量、验证可靠性与资源边界。循环策略执行过程中产生的观测、动作与环境反馈进一步形成交互轨迹，为分析现有循环策略行为提供经验依据，并支持后续演化。下一小节据此定义轨迹及其证据形式。

B. 轨迹与证据契约

循环策略的持续演化依赖于对历史运行的有效利用，这要求将执行过程形式化为连接任务、循环策略与结果的轨迹。设第 i 次运行的轨迹为

$$\tau_i = (g_i, \pi_{v(i)}, \mathcal{H}_i, y_i), \quad \mathcal{H}_i = \{(o_t, a_t, e_t)\}_{t=1}^{T_i}, \quad (4)$$

其中 g_i 表示第 i 次运行中处理的任务， $v(i)$ 表示该次运行所用已注册束的稳定版本索引， $\pi_{v(i)}$ 表示该束中的循环策略。稳定索引 $v(i)$ 可用于检索完整资产 $\mathcal{B}_{v(i)}$ ，包括其适用条件与版本来源。 $\mathcal{H}_i$ 表示由观测、动作和环境事件组成的交互历史， T_i 表示轨迹长度， y_i 表示运行结果。该表示显式保留了策略资产身份与相应交互结果之间的映射，确保历史性能能够准确归因于特定的已注册束版本。在获得可归因的轨迹后，必须进一步从中提炼能够指导循环策略调整的演化证据。本文将此类证据定义为可检验的依据，用于修改特定的循环策略组件，其来源是一组可归因、结果可验证且证据来源符合任务契约的轨迹。每条证据必须指明建议的调整方向、相关的性能变化、所涉及的策略组件以及支持该判断的原始轨迹。形式上，证据提取算子 Φ 将轨迹集 $\mathcal{D}$ 映射为

$$\mathcal{E} = \Phi(\mathcal{D}) = \{\varepsilon_j = (\rho_j, \delta_j, u_j, \mathcal{I}_j)\}_{j=1}^J, \quad (5)$$

其中 ε_j 表示演化证据的第 j 项， J 表示证据项的数量； $\rho_j \in \{+, -\}$ 表示证据的方向：正向证据表明相关行为模式与性能提升一致，可能支持循环策略复用；负向证据表明相关行为模式与失败或性能下降一致，可能支持循环策略修订； δ_j 表示相对于对照运行或可比迹的统计参考的度量变化； u_j 表示证据所针对的循环策略组件，其取值对应于观察、规划、记忆、动作、验证、恢复，

停止，或如下一小节所定义的预算控制；且 $\mathcal{I}_j \subseteq \{i \mid \tau_i \in \mathcal{D}\}$ 表示支持该判断的全局迹索引集合。

总之，迹与证据契约的转换

历史交互转化为可归因且可验证的演化证据，用于调整循环策略，为下一小节定义可修改且可复用的循环策略对象奠定基础。

C. 循环策略

运行证据必须作用于显式的循环控制对象，才能转化为可复用的循环策略经验。循环工程将工程范围扩展至外部智能体循环，并系统性地设计其目标、反馈、验证、停止及跨运行经验[16]。基于前述抽象类型声明，本文将控制智能体在完整执行循环中推进方式的外部规则集合定义为循环策略，其八元组结构表示为

$$\pi = \langle \pi_{\text{obs}}, \pi_{\text{plan}}, \pi_{\text{mem}}, \pi_{\text{act}}, \pi_{\text{ver}}, \pi_{\text{rec}}, \pi_{\text{stop}}, \pi_{\text{bud}} \rangle \in \Pi, \quad (6)$$

其中 Π 表示循环策略空间； π_{obs} 控制观测的获取与过滤， π_{plan} 控制计划的生成与更新， π_{mem} 控制记忆的写入、检索与使用， π_{act} 控制动作与工具选择， π_{ver} 控制结果验证， π_{rec} 控制失败恢复， π_{stop} 控制循环的停止条件， π_{bud} 控制资源预算的分配与调整。这八个组件共同决定了式(2)中的状态更新关系 f_π 和动作决策关系 h_π ，使循环策略能够覆盖从将环境反馈纳入控制状态到产生下一个动作的完整决策过程。图2以YAML风格格式展示了一个简化的循环策略示例。这种表示将八类循环规则组织成可读且可修改的外部结构，使循环策略内容能够独立于任何单次运行上下文进行存储、比较和复用。为支持跨任务复用循环控制经验，我们将循环策略设计为独立于特定宿主代理的外部资产，并使用统一表示承载其控制语义。循环策略可通过适配接口作为可插拔对象集成到不同的代理框架中，其内容不绑定于特定宿主实现，从而支持跨代理框架的循环策略替换、比较与演化。这种外部化设计为下一小节中的版本化资产表示提供了基础。

D. 循环策略的资产表示

作为一种可独立比较和复用的控制对象，循环策略还需要稳定的资产表示。跨运行复用循环策略要求循环策略本身、其适用条件及其版本来源能够被联合识别。我们将资产表示为

loop-policy-v2.1

```

observation: select task-relevant feedback
planning: update plan after each observation
memory: retrieve reusable trace evidence
action: choose tool or agent action
verification: require external evidence
recovery: retry | reroute | replan
stopping: verified | budget exhausted
budget control: allocate calls, time, and tokens

```

图2. 循环策略的YAML风格示例。粗体字段对应八类循环控制规则：观测、规划、记忆、动作、验证、恢复、停止和预算控制。

对应于第 v 个已注册束版本的资产，作为一个束：

$$\mathcal{B}_v = \langle \pi_v, \chi_v, \ell_v \rangle, \quad (7)$$

其中 v 是已注册束的稳定版本索引， π_v 表示该版本的循环策略， $\chi_v \subseteq \mathcal{Z}$ 表示该循环策略的适用条件集合， ℓ_v 表示版本溯源，包括直接父版本的稳定索引 p_v 以及该版本所依据的演化证据；初始束具有 $p_v = \perp$ 且无支持的演化证据。这三个组成部分共同描述了使用哪个循环策略、在何种条件下执行以及其版本溯源，使循环策略能够作为完整对象被存储和复用。注册在形成有效候选束时分配稳定索引，以便其进入评估历史和谱系；注册本身并不意味着候选已通过门控、已发布或已激活。此后， k 保留用于演化轮次或离线生成索引，不再作为稳定版本标识。束在共享适用条件 $z \in \chi_{\text{par}} \cap \chi_{\text{cand}}$ 下进行比较，其中下标 par 和 cand 分别表示父版本和候选版本， π_{par} 和 π_{cand} 为对应的循环策略。对于每个具有 $p_v \neq \perp$ 的非初始版本，版本溯源 ℓ_v 将其链接到直接父版本以及支持变更的运行证据。从父版本到后继版本的有向关系形成策略谱系，使历史经验能够沿版本关系积累，并支持后续的候选比较、版本选择和结果追踪。本节从长时程复杂任务开始，依次定义了循环交互过程、轨迹与演化证据、循环策略及其资产表示。这些定义明确了自演化的对象、经验的来源以及比较的基础，为下一节介绍在线与离线演化、候选生命周期和稳健发布门控提供了统一符号。

四、开放循环演化方法

第三节正式定义了长时程复杂任务、轨迹、演化证据和循环策略。基于这些定义，OpenLoopEvolve 将跨多次运行累积的轨迹转化为修改循环策略的基，并优化外部循环策略而非基础模型的参数。从当前冠军开始，该方法构建证据、生成候选、执行配对评估，并应用受限发布以产生后继版本。根据证据来源和更新时机的不同，这一通用框架可实例化为在线演化或离线演化。

A. OpenLoopEvolve 总体框架

OpenLoopEvolve 的两种演化模式共享从运行经验到策略资产的同一起点。设 $\mathcal{D}_k$ 表示第 k 轮演化中使用的轨迹集合。相应的证据构建过程为

$$\mathcal{E}_k = \Phi(\mathcal{D}_k), \quad \mathcal{D}_k \in \{\mathcal{D}_k^{\text{off}}, \mathcal{D}_k^{\text{on}}\}, \quad (8)$$

其中 Φ 由式 (5) 得出； $\mathcal{D}_k^{\text{off}}$ 和 $\mathcal{D}_k^{\text{on}}$ 分别是可从档案和在线运行中获取的可归因轨迹。在线模式保留轮次索引并使用 $\mathcal{E}_k = \Phi(\mathcal{D}_k^{\text{on}})$ ；因为离线档案在各代之间保持不变， $\Phi(\mathcal{D}_k^{\text{off}})$ 在下文中简称为 $\mathcal{E}$ 。仅当轨迹的结果已由 $\mathcal{D}_k$ 验证且其来源属于 $\mathcal{V}_{g_i}$ 时，该轨迹才能进入 $\mathcal{S}_{g_i}$ 。候选生成读取 $\varepsilon_j = (\rho_j, \delta_j, u_j, \mathcal{I}_j)$ ，其中 u_j 标识式 (6) 中的一个分量；

$G_{\text{LLM}}^{\text{on}}$ 返回一个已注册的束，而 $G_{\text{LLM}}^{\text{off}}$ 和 $V_{\text{LLM}}^{\text{off}}$ 返回已注册束的集合；未能形成有效束的提案产生空值或被排除在集合之外。候选只有在通过配对评估和门控之后，才可能作为后继资产发布。宿主仅在任务边界激活已发布的策略，并且只有被激活的策略根据式 (2) 控制环境交互，并按式 (4) 产生新轨迹，从而形成图 1 所示的闭环。为便于阅读，表 I 按任务与交互、轨迹与证据、策略资产以及演化与发布的顺序，总结了问题表述和方法中使用的关键符号。仅在单个方程或算法中使用的索引、临时变量和超参数在局部定义。冠军-挑战者配对评估为两种模式 [18]、[19] 提供了共同的比较基础。设当前冠军束为 $\mathcal{B}_c = \langle \pi_c, \chi_c, \ell_c \rangle$ ，在第 j 轮生成的候选束 k 为 $\mathcal{B}_{k,j} = \langle \pi_{k,j}, \chi_{k,j}, \ell_{k,j} \rangle \equiv \mathcal{B}_{v_{k,j}}$ ，其中 (k, j) 是候选的搜索坐标， $v_{k,j}$ 是候选形成时分配的稳定版本索引。两者仅在同一任务 g_i 和相同的受控运行条件 $z_{i,j} \in \chi_c \cap \chi_{k,j}$ 下分别执行。对于候选 i 的配对 j ，价值得分差和相对变化定义为

$$\Delta_{i,j} = q(\pi_{k,j}; g_i, z_{i,j}) - q(\pi_c; g_i, z_{i,j}), \quad (9)$$

$$r_{i,j} = \frac{\Delta_{i,j}}{\max\{|q(\pi_c; g_i, z_{i,j})|, \epsilon\}},$$

其中， $\Delta_{i,j}$ 是候选方案相对于冠军方案的绝对值分数变化， $r_{i,j}$ 是相对变化， $\epsilon > 0$ 在冠军值分数接近零时确保数值稳定性。在每次配对运行中，对应束中的循环策略控制式 (2) 中的执行循环。值分数 q 仍由任务契约中的 $\mathcal{M}_{g_i}$ 和 $\mathcal{V}_{g_i}$ 决定，而动作范围和资源上限分别遵守 $\mathcal{U}_{g_i}$ 和 β_{g_i} 。保持任务、模型、工具和资源条件不变，可控制主要外部变异来源，使配对变化更直接地反映循环策略修改的效果。候选方案 j 的有效相对变化构成样本 $r_j = \{r_{i,j}\}$ ，并与式 (3) 中的验证风险 $\mathbf{c}_{\mathcal{V}_{g_i}}$ 、资源消耗 $\mathbf{b}$ 、失败状态和证据有效性一起，构成配对评估 $\mathcal{P}_{k,j}$ 。对于出现终止失败或无效证据的运行，评估协议采用有界失败感知变化，并保留相应的有效性和可靠性状态。因此，下文算法中的 PairEval ($\mathcal{B}_c, \mathcal{B}_{k,j}$) 始终表示在共享适用条件下对完整束的配对评估，而非不考虑适用条件地对两个裸策略进行比较。配对评估并不根据单一平均分数直接替换当前策略；它为受限发布提供证据。令 Γ_b 、 Γ_e 、 Γ_t 和 Γ_c 分别表示收益、证据质量、尾部风险和资源成本四类约束是否得到满足。发布门可表示为

$$\text{Gate}(\mathcal{P}_{k,j}, \Theta) = \text{accept} \iff \prod_{\alpha \in \mathcal{J}_{\text{gate}}} \Gamma_{\alpha}(\mathcal{P}_{k,j}, \Theta_{\alpha}) = 1, \quad (10)$$

其中 $\mathcal{J}_{\text{gate}} = \{b, e, t, c\}$ 分别对应收益、证据质量、尾部风险与资源成本； $\Gamma_{\alpha}(\mathcal{P}_{k,j}, \Theta_{\alpha}) \in \{0, 1\}$ 表示类别 α 的约束是否得到满足； $\Theta = \{\Theta_{\alpha}\}$ 为门控配置。 Γ_e 检查 $\mathcal{S}_{g_i}$ ，所允许的验证， Γ_t 强制执行 $\mathbf{c}_{\mathcal{V}_{g_i}} \preceq \kappa_{g_i}$ ， Γ_c 强制执行 $\mathbf{b} \preceq \beta_{g_i}$ 。

配对数量、胜率、置信区间、任务失败次数及尾部损失可作为内部评估量。仅当所有已启用约束均得到满足时，挑战者才具备发布资格，且配置的阈值不得放宽任务契约中的验证与资源边界。在此通用框架内，在线与离线模式分别适用于不同形式的可用运行经验。当任务分布持续变化或需及时纳入新反馈时，在线模式可进行分阶段更新。当归档迹覆盖目标任务的主要运行状态时，离线模式可对多个策略方向进行集中搜索。应用根据可用迹的范围、反馈及时性与更新需求选择模式；系统不会自动切换模式。两种模式在候选搜索过程与更新时机上存在差异，而其策略比较与发布资格统一由式 (9) 和式 (10) 约束。两种模式的评估历史均被显式类型化。令 $\mathcal{R} = \mathcal{R}^{\text{on}} \cup \mathcal{R}^{\text{off}}$ 为不相交并集。每条记录

表 I 主要符号及含义

Category	Symbol	Meaning
Task and interaction	$g = \langle d, \mathcal{U}_g, \mathcal{M}_g, \mathcal{S}_g, \mathcal{V}_g, \beta_g \rangle$	Executable contract for a long-horizon task, comprising its objective, permitted executors, evaluation metrics, evidence sources, verification protocol, and resource budget
	$x_t, o_t, a_t, e_t; f_\pi, h_\pi$	Control state, observation, action, and environment event at step t ; state-update and action-decision functions constrained by the policy
	$z \in \mathcal{Z}; q(\pi; g, z); \mathbf{c}_{\mathcal{V}_g}, \kappa_g; \mathbf{b}, \beta_g$	Controlled run condition; value score; verification risk and its upper bound; resource consumption and its upper bound
Traces and evidence	$\tau_i = (g_i, \pi_{v(i)}, \mathcal{H}_i, y_i)$	Attributable trace of run i , including the task, stable Bundle version, interaction history, and run outcome
	$\mathcal{D}, \mathcal{E} = \Phi(\mathcal{D})$	Set of attributable traces with verified outcomes and compliant evidence sources; the evolution-evidence set; and the evidence-extraction operator
	$\varepsilon_j = (\rho_j, \delta_j, u_j, \mathcal{I}_j)$	Direction, metric change, target policy component, and supporting trace indices of evolution-evidence item j
Policy assets	$\pi \in \Pi$	Loop Policy and its policy space; the policy comprises rules for observation, planning, memory, action, verification, recovery, stopping, and budget control
	$\mathcal{B}_v = \langle \pi_v, \chi_v, \ell_v \rangle$	Bundle corresponding to stable version index v , comprising a Loop Policy, a set of applicability conditions, and version provenance
	χ_v, ℓ_v	Bundle applicability conditions and version provenance containing the parent version and supporting evidence
Evolution and release	$\mathcal{D}^{\text{off}}, \mathcal{D}_k^{\text{on}}; \mathcal{E}, \mathcal{E}_k$	Archived offline traces, traces visible in online round k , and their corresponding evolution evidence
	$G_{\text{LLM}}^{\text{on}}, G_{\text{LLM}}^{\text{off}}, V_{\text{LLM}}^{\text{off}}$	Online single-candidate generation, offline initial-candidate-set generation, and offline candidate mutation and recombination operators
	$\mathcal{R} = \mathcal{R}^{\text{on}} \uplus \mathcal{R}^{\text{off}}$	Mode-typed evaluation histories; the online component records candidates, evaluations, and release decisions, whereas the offline component records candidate sets and evaluation mappings
	$\mathcal{B}_c, \mathcal{B}_{k,j} \equiv \mathcal{B}_{v_{k,j}}$	Champion and Challenger Bundle j in round k ; $v_{k,j}$ is its stable version index (online: $j = 1, \mathcal{B}'_k \equiv \mathcal{B}_{k,1}$)
	$\Delta_{i,j}, r_{i,j}, \mathcal{P}_{k,j}$	Absolute and relative value-score changes for pair i , and the complete paired evaluation of a candidate
	$\Gamma_\alpha, \Theta_\alpha, \text{Gate}$	Release constraint of category α , its configuration, and the aggregate release-gate operator
	$\mathcal{C}_k, \mathcal{F}_k, \overline{\mathcal{P}}_k$	Candidate set, retained set comprising the fixed Champion and selected nondominated candidates, and cumulative paired-evaluation mapping at offline generation k

在线组件 $\mathcal{R}^{\text{on}}$ 中的记录为 $(\mathcal{B}', \mathcal{P}, \omega)$ ，分别存储候选束、其配对评估及发布决策。离线生成 k 在 $\mathcal{R}^{\text{off}}$ 中的记录为 $(\mathcal{C}_k, \mathcal{P}_k)$ ，存储候选集及其评估映射。每个生成器仅读取自身模式的历史组件，从而避免将两种记录结构解释为相同的无类型元组。

B. 在线循环策略演化

在线演化利用持续的最新反馈流分阶段更新循环策略，使后续任务能够纳入最新的运行经验。在在线更新时间 k ，宿主提交可归因于显式策略版本的已完成迹。一旦满足应用定义的更新条件，这些迹便转化为演化证据 $\mathcal{E}_k$ 。随后，大语言模型生成挑战者循环策略，并将其封装在候选束中：

其中 $\mathcal{B}_k = \langle \pi_k, \chi_k, \ell_k \rangle$ 是当前在线更新时刻保持的冠军， k ，而 $\mathcal{R}_{<k}^{\text{on}}$ 是之前的在线评估历史；此处 k 表示更新时刻，而非稳定版本索引。在线模式每轮有一个候选，为此我们设置 $j = 1$ 。 $G_{\text{LLM}}^{\text{on}}$ 次大语言模型重写 π'_k ，设置 $\chi'_k \subseteq \chi_k$ ，并在 ℓ'_k 中记录当前轮的父版本和证据。当有效候选形成时，稳定版本索引 $v_{k,1}$ 被原子地分配，候选被写为 $\mathcal{B}'_k \equiv \mathcal{B}_{k,1} \equiv \mathcal{B}_{v_{k,1}}$ ，从而直接产生如式 (7) 所定义的已注册候选束。该候选仅在 $\chi_k \cap \chi'_k$ 内进行配对评估。若通过门控，则被记录为已发布的继任资产，并在后续任务边界激活为 $\mathcal{B}_{k+1} = \mathcal{B}'_k$ ；否则， $\mathcal{B}_{k+1} = \mathcal{B}_k$ 。为强调单次更新的处理顺序，算法 1 在循环体内省略了更新下标 k ； $\mathcal{B}, \mathcal{D}^{\text{on}}, \mathcal{E}$ 和 $\mathcal{R}'$ 分别对应当前批次的 $\mathcal{B}_k, \mathcal{D}_k^{\text{on}}, \mathcal{E}_k$ 和 $\mathcal{B}'_k$ 。

$$\mathcal{B}'_k = G_{\text{LLM}}^{\text{on}}(\mathcal{B}_k, \mathcal{E}_k, \mathcal{R}_{<k}^{\text{on}}) = \langle \pi'_k, \chi'_k, \ell'_k \rangle, \quad (11)$$

在线更新必须在每个任务内保持策略一致性，并支持发布后的回滚。一个任务是并

从头到尾被同一个已激活的冠军（Champion）所困扰。通过门控的挑战者（Challenger）可以被释放，但仅在后续任务边界处被激活，因此运行结果仍可归因于明确的策略版本。新版本激活后，金丝雀监控器（CANARYMONITOR）利用可归因的后续反馈监控其性能。当满足退化条件时，它回滚到父版本，并将触发回滚的迹（trace）从当前更新批次中隔离；同一批次中的其他可用迹仍可参与候选生成。更新条件 Trig 可能取决于反馈量、更新周期或任务阶段。其唯一功能是确定进化何时开始；它并不取代整体框架中的发布门控。因此，在线模式形成了反馈触发、候选验证、发布、后续任务激活和退化回滚的闭环，详见算法 1。

Algorithm 1 Online Loop Policy Optimization

Input: Champion Bundle $\mathcal{B} = \langle \pi_c, \chi, \ell \rangle$, stream of attributable traces $\{\tau_i\}$ (with y_i verified by $\mathcal{V}_{g_i}$ and its evidence source belonging to $\mathcal{S}_{g_i}$), update condition Trig, online evaluation history $\mathcal{R}^{\text{on}}$, gate configuration Θ

Output: Champion $\mathcal{B}$ updated with the trace stream and online evaluation history $\mathcal{R}^{\text{on}}$

- ▷ Continuously receive batches of traces submitted by the host when checkpoint or feedback conditions are met.
- 1: **while** new trace batches continue to arrive **do**
- 2: $\mathcal{D}^{\text{on}} \leftarrow \text{RECEIVETRACES}$
 - ▷ Monitor the performance of the new version; roll back a degraded version, and let $\tilde{\mathcal{D}}^{\text{on}}$ denote the usable traces after quarantining the corresponding feedback.
- 3: $(\mathcal{B} = \langle \pi_c, \chi, \ell \rangle, \tilde{\mathcal{D}}^{\text{on}}) \leftarrow \text{CANARYMONITOR}(\mathcal{B}, \mathcal{D}^{\text{on}})$
 - ▷ Start evolution only when usable feedback exists and the external update condition is satisfied.
- 4: **if** $\tilde{\mathcal{D}}^{\text{on}} = \emptyset$ **or** $\neg \text{Trig}(\tilde{\mathcal{D}}^{\text{on}})$ **then continue**
 - ▷ Derive the evolution evidence for the current round from recent traces.
- 5: $\mathcal{E} \leftarrow \Phi(\tilde{\mathcal{D}}^{\text{on}})$
 - ▷ Use the large language model to form a valid registered Challenger Bundle; registration atomically assigns its stable index (online $j = 1$) before evaluation or history insertion.
- 6: $\mathcal{B}' = \langle \pi', \chi', \ell' \rangle \leftarrow G_{\text{LLM}}^{\text{on}}(\mathcal{B}, \mathcal{E}, \mathcal{R}^{\text{on}})$
 - ▷ Wait for the next feedback batch if no testable candidate is formed.
- 7: **if** $\mathcal{B}' = \emptyset$ **then continue**
 - ▷ Perform paired evaluation of the Bundles under shared conditions in $\chi \cap \chi'$, and let ω denote the release-gate decision.
- 8: $\mathcal{P} \leftarrow \text{PAIREVAL}(\mathcal{B}, \mathcal{B}')$, $\omega \leftarrow \text{GATE}(\mathcal{P}, \Theta)$
 - ▷ Record the candidate, evaluation result, and release decision for the current round.
- 9: $\mathcal{R}^{\text{on}} \leftarrow \mathcal{R}^{\text{on}} \cup \{(\mathcal{B}', \mathcal{P}, \omega)\}$
 - ▷ Mark a gate-passing candidate as the released successor asset and activate it as the current Champion only at a subsequent task boundary.
- 10: **if** $\omega = \text{accept}$ **then** $\mathcal{B} \leftarrow \mathcal{B}'$
- 11: **end while**
 - ▷ Return the policy asset and evaluation history retained after online updates.
- 12: **return** $\mathcal{B}, \mathcal{R}^{\text{on}}$

C. 离线环路策略演化

离线演化以归档轨迹和当前冠军为输入，在任务执行路径之外执行多候选、多代的评估—选择—生成搜索，并释放一个通过发布门限的候选束作为继任冠军。归档轨迹 $\mathcal{D}^{\text{off}}$ 通过式 (8) 产生演化证据 $\mathcal{E}$ 。基于这些证据，大语言模型围绕冠军自主提出候选，并根据前几代的评估更新搜索方向。为了在搜索过程中保持式 (7) 所定义的策略身份、适用条件和版本来源，候选不以裸策略形式存储。

设 K 为演化总代数， $\mathcal{C}_k$ 为第 k ， $\mathcal{F}_k$ 代候选束集合， k 为截至该代保留的固定冠军与所选非支配候选组成的保留集， $\bar{\mathcal{P}}_k$ 为截至该代的累积配对评估映射。则离线搜索可写为

$$\begin{aligned}
 \mathcal{F}_0 &= \{\mathcal{B}\}, \\
 \bar{\mathcal{P}}_0 &= \emptyset, \\
 \mathcal{C}_1 &= G_{\text{LLM}}^{\text{off}}(\mathcal{B}, \mathcal{E}, \mathcal{R}_0^{\text{off}}), \\
 \mathcal{P}_k &= \{\mathcal{B}_{k,j} \mapsto \mathcal{P}_{k,j}\}_{\mathcal{B}_{k,j} \in \mathcal{C}_k}, \\
 \mathcal{P}_{k,j} &= \text{PairEval}(\mathcal{B}, \mathcal{B}_{k,j}), \\
 \bar{\mathcal{P}}_k &= \bar{\mathcal{P}}_{k-1} \cup \mathcal{P}_k, \\
 \mathcal{A}_k &= (\mathcal{F}_{k-1} \setminus \{\mathcal{B}\}) \cup \mathcal{C}_k, \\
 \mathcal{F}_k &= \{\mathcal{B}\} \cup \text{Select}(\mathcal{A}_k, \bar{\mathcal{P}}_k), \\
 \mathcal{C}_{k+1} &= V_{\text{LLM}}^{\text{off}}(\mathcal{F}_k, \mathcal{E}, \mathcal{R}_k^{\text{off}}).
 \end{aligned} \tag{12}$$

其中 $\mathcal{B} = \langle \pi_c, \chi_c, \ell_c \rangle$ 为固定的冠军（Champion）； $\mathcal{P}_k$ 为当前代的评估映射，以候选束（Bundle）为键，每个候选评估 $\mathcal{P}_{k,j}$ 为其值； $\bar{\mathcal{P}}_k$ 为累积评估映射，通过将每一代合并到初始为空的映射 $\bar{\mathcal{P}}_0$ 中获得； $\mathcal{R}_k^{\text{off}}$ 为离线评估历史。每个形成的候选都根据 $\mathcal{B}_{k,j} \equiv \mathcal{B}_{v_{k,j}}$ ，so 注册一个稳定的版本索引，合并映射不会混淆不同代的束。 $\mathcal{A}_k$ 收集前几代保留的候选和当前代的新候选，但不包括固定的冠军。因此，选择（Select）仅查询评估映射域中的候选，之后它们与 $\mathcal{B}$ 的并集形成 $\mathcal{F}_k$ 。 V^{off} 。当 $k < K$ 时，大语言模型（LLM）生成下一代。在第 j 代通过重组产生的候选 k 在 $\ell_{k,j}$ 中指定一个直接父版本，而其他供体记录在演化证据中；其适用条件仅限于直接父版本的适用条件和证据覆盖范围。这种关系保留了跨代的有效搜索方向及其来源。配对评估为代际更新提供反馈。每一代的候选束在适用条件与 χ_c 的交集内，与冠军在共享任务和运行条件下进行比较。价值分数、验证风险和资源消耗的定义继续遵循第三节中的任务契约和公式 (3)。收益、可靠性、尾部性能和资源成本共同决定保留。大语言模型相应地修改控制规则或重组来自不同策略的互补规则，而被淘汰候选暴露的失败也进入评估历史。冠军保持稳定参考，使成功和失败共同约束下一代的搜索方向。跨代的非支配保留仅决定后续搜索；正式发布仍受公式 (3) 和 (10) 约束。经过 K 代演化后，令 $\mathcal{F}_K^{\text{acc}}$ 表示最终保留集中有资格发布的候选束。价值选择（ValueSelect）选择候选

with the highest value score aggregated by the evaluation protocol, using reliability, tail performance, and resource cost for nondominated tie-breaking only when value scores are tied, and makes it the successor Champion $\mathcal{B}^+ = \mathcal{B}^*$. If $\mathcal{F}_K^{\text{acc}}$ is empty, the original Champion remains unchanged. Algorithm 2 gives the procedure.

Algorithm 2 Offline Loop Policy Optimization

Input: Champion Bundle $\mathcal{B} = \langle \pi_c, \chi_c, \ell_c \rangle$, archived traces $\mathcal{D}^{\text{off}} = \{\tau_i\}$ (with y_i verified by $\mathcal{V}_{g_i}$ and its evidence source belonging to $\mathcal{S}_{g_i}$), initial offline evaluation history $\mathcal{R}_0^{\text{off}}$, number of evolution generations K , gate configuration Θ

Output: Successor Champion $\mathcal{B}^+$ and offline evaluation history $\mathcal{R}_K^{\text{off}}$ after evolution

```

1:  $\mathcal{E} \leftarrow \Phi(\mathcal{D}^{\text{off}})$ 
   ▷ Derive evidence related to Loop Policy modifications from archived runs.
   ▷ Use the large language model to form the initial valid candidate Bundle set; each candidate is atomically assigned stable index  $v_{1,j}$  before entering the evaluation mapping.
2:  $\mathcal{C}_1 \leftarrow G_{\text{LLM}}^{\text{off}}(\mathcal{B}, \mathcal{E}, \mathcal{R}_0^{\text{off}})$ 
   ▷ Initialize the elite set and cumulative evaluations with the complete Champion Bundle.
3:  $\mathcal{F}_0 \leftarrow \{\mathcal{B}\}, \bar{\mathcal{P}}_0 \leftarrow \emptyset$ 
   ▷ Advance the multi-generation search through paired evaluation and evidence-guided policy generation.
4: for  $k = 1, \dots, K$  do
   ▷ Perform Bundle paired evaluation for each candidate under applicability conditions shared with  $\chi_c$ , and construct the evaluation mapping for the current generation.
5:    $\mathcal{P}_k \leftarrow \emptyset$ 
6:   for all  $\mathcal{B}_{k,j} \in \mathcal{C}_k$  do
7:      $\mathcal{P}_k(\mathcal{B}_{k,j}) \leftarrow \text{PAIREVAL}(\mathcal{B}, \mathcal{B}_{k,j})$ 
8:   end for
   ▷ Accumulate the evaluations for the current generation and retain competitive policy directions from existing elite Bundles and new candidates.
9:    $\mathcal{R}_k^{\text{off}} \leftarrow \mathcal{R}_{k-1}^{\text{off}} \cup \{(\mathcal{C}_k, \mathcal{P}_k)\}, \bar{\mathcal{P}}_k \leftarrow \bar{\mathcal{P}}_{k-1} \cup \mathcal{P}_k$ 
10:   $\mathcal{A}_k \leftarrow (\mathcal{F}_{k-1} \setminus \{\mathcal{B}\}) \cup \mathcal{C}_k$ 
11:   $\mathcal{F}_k \leftarrow \{\mathcal{B}\} \cup \text{SELECT}(\mathcal{A}_k, \bar{\mathcal{P}}_k)$ 
12:  if  $k < K$  then
   ▷ Use the large language model to mutate and recombine policies from retained Bundles and cumulative feedback; each newly formed valid Bundle is assigned stable index  $v_{k+1,j}$  before next-generation evaluation.
13:     $\mathcal{C}_{k+1} \leftarrow V_{\text{LLM}}^{\text{off}}(\mathcal{F}_k, \mathcal{E}, \mathcal{R}_k^{\text{off}})$ 
14:  end if
15: end for
   ▷ Collect candidates from the final retained Bundles that pass the robust release gate.
16:  $\mathcal{F}_K^{\text{acc}} \leftarrow \{\mathcal{B}' \in \mathcal{F}_K \setminus \{\mathcal{B}\} \mid \text{GATE}(\bar{\mathcal{P}}_K(\mathcal{B}'), \Theta) = \text{accept}\}$ 
   ▷ Retain the original Champion if no candidate is eligible for release.
17: if  $\mathcal{F}_K^{\text{acc}} = \emptyset$  then  $\mathcal{B}^+ \leftarrow \mathcal{B}$  and return  $\mathcal{B}^+, \mathcal{R}_K^{\text{off}}$ 
   ▷ Select the qualified Bundle with the highest aggregate value and release it in full as the successor version of the Champion.
18:  $\mathcal{B}^* \leftarrow \text{VALUESELECT}(\mathcal{F}_K^{\text{acc}}, \bar{\mathcal{P}}_K), \mathcal{B}^+ \leftarrow \mathcal{B}^*$ 
19: return  $\mathcal{B}^+, \mathcal{R}_K^{\text{off}}$ 

```

五、实验结果

A. YC-Bench 结果

我们利用 YC-Bench [6] 评估 OpenLoopEvolve 对长时程任务执行的影响。该基准要求智能体在模拟的一年内持续运营一家企业，并以最终资金衡量顺序决策的累积效应，包括任务选择、资源分配和风险管理。我们将使用官方 YC-Bench 智能体循环但既不集成 OpenLoopEvolve 也不加载额外循环策略的原生智能体指定为基线，从而衡量模型在该基准原生执行框架下的能力。为了分离引入策略与演化策略的效果，固定- π_0 加载与两种 OLE 模式相同的初始循环策略，

但在整个年度内保持不变。OLE-online 从 π_0 开始，在执行过程中在线演化并更新策略，共进行 12 次候选更新尝试；OLE-offline 使用实验前通过离线演化获得的固定策略，评估期间不更新。每次触发时，离线演化使用大语言模型自主生成 $k=3$ 个候选，预注册的最大候选轮数为 4 轮。四种设置均使用 deepseek-v4-flash [20]，采用官方中等配置，种子为 1/2/3，上下文为 20 轮，并从相同的初始状态开始年度评估。表 II 总结了四种设置的年度结果。相对于固定- π_0 ，OLE-online 和 OLE-offline 的平均最终资金分别提高 140.07% 和 166.31%，任务成功率分别提高 13.98 和 17.92 个百分点。年度存活率从 1/3 提高到 2/3 和 3/3，而平均最大回撤分别降低 10.77% 和 38.30%。这一比较表明，在线和离线演化都能从初始循环策略产生更有效的循环策略，在长期回报、任务成功率、年度存活率和风险度量方面具有一致的优势。图 3 展示了四种设置在模拟年度内的资金轨迹。

B. 令牌使用与演化成本

为衡量引入在线学习进化 (OLE) 后推理资源的变化，本文汇总每次模型调用记录的输入与输出令牌，并将其总和作为令牌使用量。四种设置均包含官方种子 1/2/3。基线与固定 π_0 仅生成主任务执行调用，而在线学习进化在线与在线学习进化离线还通过配对评估产生候选-冠军进化验证调用。表 III(a) 报告主任务令牌、平均存活时长及每次调用的平均令牌使用量，并计算相对于基线和固定 π_0 的平均变化。表 III(b) 比较四种设置下的主任务总量、进化验证成本及可观测总量。基线、固定 π_0 、在线学习进化在线与在线学习进化离线的主任务执行分别包含 1108、799、1575 和 1907 次模型调用，平均存活时长分别为 243.00、207.78、294.00 和 365.00 天。相应的每次调用平均使用量为 25001、23496、22029 和 18676 个令牌。相对于基线，在线学习进化在线与在线学习进化离线的平均使用量分别下降 11.89% 和 25.30%；相对于固定 π_0 ，分别下降 6.24% 和 20.51%。尽管两种在线学习进化模式的主任务令牌总量更高，但它们也支持更长的执行周期和更多的任务调用，同时每次调用的平均令牌使用量更少。这表明，在本实验中，循环策略在控制每次交互推理成本的同时，扩大了持续任务执行的规模。在线模式尝试 12 个候选，进化验证成本为 29.82M 个令牌，加上主任务使用量后，可观测总量为 64.52M 个令牌。离线模式的进化验证成本为 24.02M 个令牌，可观测总量为 59.63M 个令牌。基线和固定 π_0 不执行进化，因此其可观测总量等于其

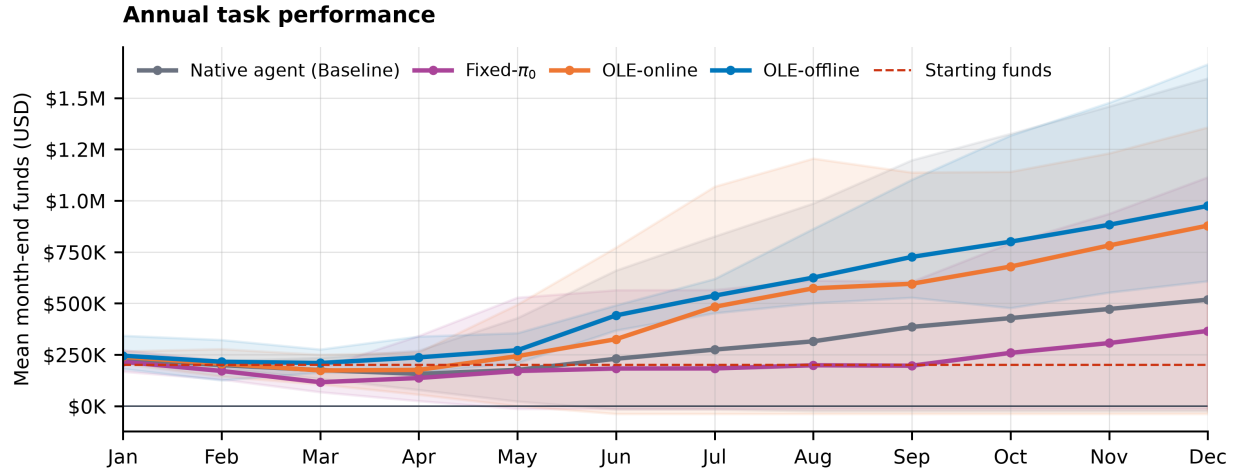


图 3. 四种设置在 YC-Bench 上的年度任务表现。曲线和阴影区域分别表示三个种子下的平均月末资金和跨种子范围；提前终止后，期末资金仅被结转以保留固定的聚合分母，并不表示继续执行。

表 II YC-BENCH 上的总体结果。

Method	Mean final funds (\$)	Δ_{Native}	Δ_{π_0}	Task success rate	Annual survival	Mean maximum drawdown (\$)
Native agent (Baseline)	518,158.46	—	—	77.23%	1/3	191,462.39
Fixed initial policy (Fixed- π_0)	365,976.30	-29.37%	—	73.89%	1/3	187,022.86
OLE-online	878,601.79	+69.56%	+140.07%	87.87%	2/3	166,878.69
OLE-offline (fixed policy)	974,627.52	+88.09%	+166.31%	91.80%	3/3	115,402.09

Δ_{Native} 和 Δ_{π_0} 分别使用原生智能体和固定初始策略作为各自的参考。

表 III 在线学习进化的主任务令牌使用量与进化验证成本。

(a) Main-task execution					
Method	Total (M tokens)	Mean survival duration (days)	Average per call (tokens)	Δ_{Base} (%)	Δ_{Fixed} (%)
Baseline	27.70	243.00	25,001	—	—
Fixed- π_0	18.77	207.78	23,496	-6.02%	—
OLE-online	34.70	294.00	22,029	-11.89%	-6.24%
OLE-offline	35.61	365.00	18,676	-25.30%	-20.51%

(b) Main-task and evolution costs			
Method	Main task (M tokens)	Evolution (M tokens)	Total (M tokens)
Baseline	27.70	0.00	27.70
Fixed- π_0	18.77	0.00	18.77
OLE-online	34.70	29.82	64.52
OLE-offline	35.61	24.02	59.63

(a) “总计”为三个官方种子中主任务令牌的总和，单位为百万；“平均存活时长”以天为单位报告；“每次调用平均”为主任务令牌使用量除以模型调用次数。 Δ_{Base} 和 Δ_{Fixed} 分别表示该平均值相对于基线和固定 π_0 的变化。

(b) 主任务总量、候选 - 冠军演化验证成本及其在四种设置下的总和，单位均为百万令牌。基线与固定 π_0 不执行演化，因此演化成本为零；OLE 总量不包括未保留用量的候选生成调用。

主任务分别使用了 27.70M 和 18.77M 个标记。鉴于持续执行和整体任务性能的提升，在当前实验规模下，额外的演化成本是可以接受的。未来工作将进一步检验循环策略是否在跨领域保持有效

不同的任务和操作环境。

六、结论

本文提出开放循环进化 (OpenLoopEvolve)，将智能体的执行循环表示为外部化的循环策略 (Loop Policy)，该策略具备版本化、可追踪和可复用的特性。该框架从运行轨迹中构建进化证据，利用大语言模型生成候选策略，并通过冠军-挑战者配对评估与稳健门控机制管理策略更新。其在线与离线模式分别利用近期反馈与归档经验，使循环改进能够超越单一情境持续累积。在 YC-Bench 上的实验表明，两种进化模式相较于固定的初始策略均能提升长时程任务性能，同时支持更长的有效执行，并控制每次交互的推理成本。这些结果揭示了循环策略作为独立资产的更广泛作用：它们可以从运行证据中持续精炼，并可在不同任务、智能体宿主及运行环境间迁移与复用。未来工作将在更大规模上评估这种跨场景有效性、进化稳定性及成本控制。

参考文献

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations*, 2023. [Online]. Available: <https://arxiv.org/abs/2210.03629>

- [2] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, "AgentBench: Evaluating LLMs as agents," in *International Conference on Learning Representations*, 2024. [Online]. Available: <https://arxiv.org/abs/2308.03688>
- [3] C. Ma, J. Zhang, Z. Zhu, C. Yang, Y. Yang, Y. Jin, Z. Lan, L. Kong, and J. He, "AgentBoard: An analytical evaluation board of multi-turn LLM agents," in *Advances in Neural Information Processing Systems*, vol. 37, 2024. [Online]. Available: <https://arxiv.org/abs/2401.13178>
- [4] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, "WebArena: A realistic web environment for building autonomous agents," in *International Conference on Learning Representations*, 2024. [Online]. Available: <https://arxiv.org/abs/2307.13854>
- [5] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, " τ -bench: A benchmark for tool-agent-user interaction in real-world domains," in *International Conference on Learning Representations*, 2025. [Online]. Available: <https://arxiv.org/abs/2406.12045>
- [6] M. He, A. Jain, A. Kumar, V. Tu, S. Bakshi, S. Patro, and N. Rajani, "YC-Bench: Benchmarking AI agents for long-term planning and consistent execution," *arXiv preprint arXiv:2604.01212*, 2026. [Online]. Available: <https://arxiv.org/abs/2604.01212>
- [7] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023. [Online]. Available: <https://doi.org/10.1145/3586183.3606763>
- [8] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html
- [9] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang, "ExpeL: LLM agents are experiential learners," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 17, 2024, pp. 19 632–19 642. [Online]. Available: <https://ojs.aaai.org/index.php/AAAI/article/view/29936>
- [10] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, "Voyager: An open-ended embodied agent with large language models," *Transactions on Machine Learning Research*, 2024. [Online]. Available: <https://openreview.net/forum?id=ehfRiFOR3a>
- [11] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. V. A., S. Haq, A. Sharma, T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts, "DSPy: Compiling declarative language model calls into self-improving pipelines," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/hash/f1cf02ce09757f57c3b93c0db83181e0-Abstract-Conference.html
- [12] S. Hu, C. Lu, and J. Clune, "Automated design of agentic systems," in *International Conference on Learning Representations*, 2025. [Online]. Available: <https://arxiv.org/abs/2408.08435>
- [13] J. Zhang, J. Xiang, Z. Yu, F. Teng, X. Chen, J. Chen, M. Zhuge, X. Cheng, S. Hong, J. Wang, B. Zheng, B. Liu, Y. Luo, and C. Wu, "AFlow: Automating agentic workflow generation," in *International Conference on Learning Representations*, 2025. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/hash/5492ecbce4439401798dcd2c90be94cd-Abstract-Conference.html
- [14] Y. Shang, Y. Li, K. Zhao, L. Ma, J. Liu, F. Xu, and Y. Li, "AgentSquare: Automatic LLM agent search in modular design space," in *International Conference on Learning Representations*, 2025. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2025/hash/0ae94013da7cd459402fd77874e09ee3-Abstract-Conference.html
- [15] Y. Guo, J. Lin, H. Wang, Y. Han, S. Hu, Z. Ni, L. Wang, and M. Chen, "SE-Agent: Self-evolution trajectory optimization in multi-step reasoning with LLM-based agents," in *Advances in Neural Information Processing Systems*, vol. 38, 2025. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2025/hash/a911e543a95493ae5004fdc01909043e-Abstract-Conference.html
- [16] S. Macedo, "Stop hand-holding your coding agent: Engineering the loops that replace step-by-step prompting," *arXiv preprint arXiv:2607.00038*, 2026. [Online]. Available: <https://arxiv.org/abs/2607.00038>
- [17] J. Huang, J. Hsia, J. Sun, F. Shi, W. Huang, and I. H. White, "Proof-or-stop: Don't trust the agent, trust the evidence—loop engineering for verifiable evidence-gated lifecycle control," *arXiv preprint arXiv:2607.14890*, 2026. [Online]. Available: <https://arxiv.org/abs/2607.14890>
- [18] S. V. Nath, "Champion-challenger based predictive model selection," in *Proceedings 2007 IEEE SoutheastCon*, 2007, pp. 254–254. [Online]. Available: <https://doi.org/10.1109/SECON.2007.342897>
- [19] Q. Wu, C. Wang, J. Langford, P. Mineiro, and M. Rossi, "ChaCha for online AutoML," in *Proceedings of the 38th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 139, 2021, pp. 11 263–11 273. [Online]. Available: <https://proceedings.mlr.press/v139/wu21d.html>
- [20] DeepSeek-AI, "DeepSeek-V4: Towards highly efficient million-token context intelligence," *arXiv preprint arXiv:2606.19348*, 2026. [Online]. Available: <https://arxiv.org/abs/2606.19348>